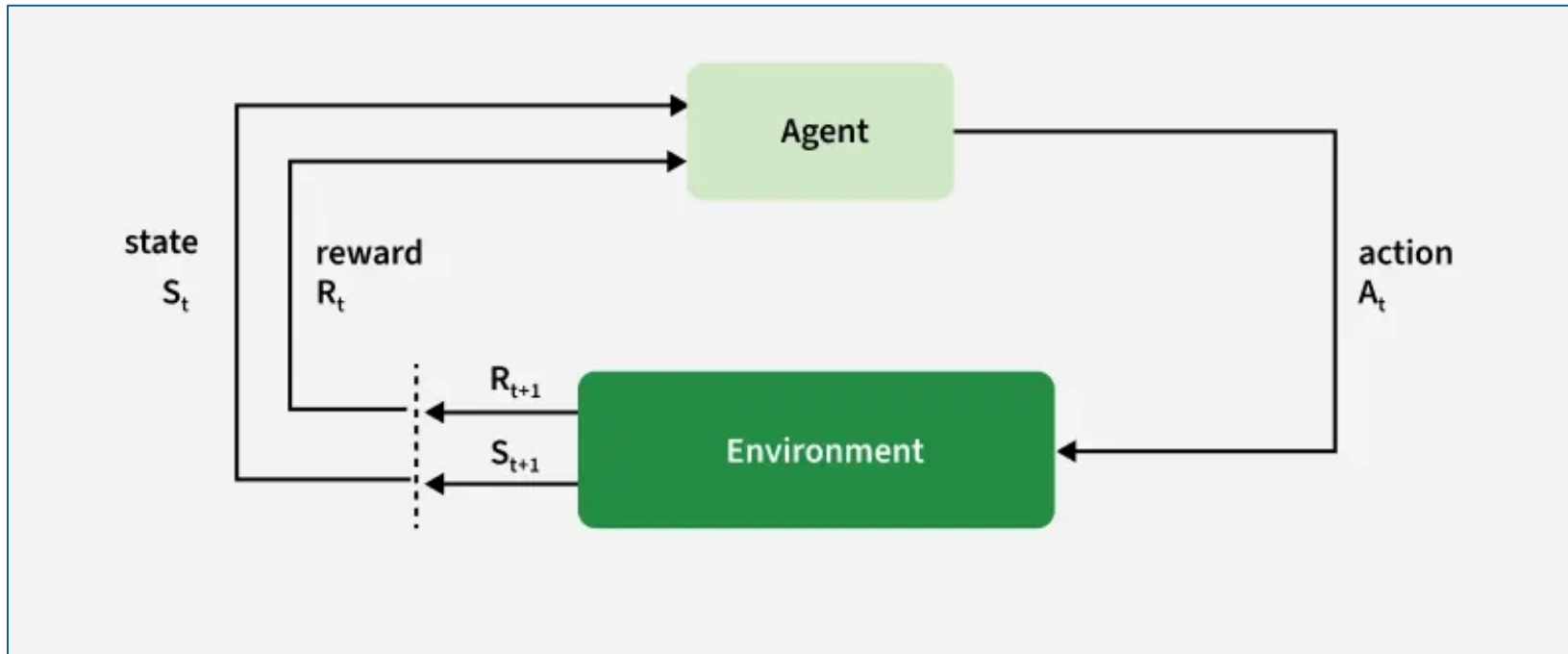


Reinforcement Learning



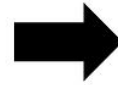
Module 3

Reinforcement Learning

Module 3

- Temporal-difference Learning,
- Temporal-difference Prediction
- Temporal-difference Control
 - Off-Policy: Q-learning
 - On-Policy: SARSA
- N-Step Bootstrapping
- Models and Planning
 - Dyna Algorithm
- Trajectory Sampling
- Real-time Dynamic Programming
- Planning at Decision Time
- Monte Carlo Tree Search

Temporal Difference Learning



Temporal Difference Learning

- Temporal Difference (TD) Learning is a method in Reinforcement Learning that learns from the raw experience.
- TD is a core model-free reinforcement learning method that learns to predict future rewards by updating value estimates based on the difference between successive predictions.
- The algorithm takes the benefits of both the Monte Carlo method and dynamic programming (DP) into account:
 - Like the Monte Carlo method, it doesn't require model dynamics, and
 - Like Dynamic Programming, it doesn't need to wait until the end of the episode to make an estimate of the value function
- Temporal difference learning approximates the current estimate based on the previously learned estimate. This approach is also called bootstrapping.

TD Learning: State Value Function

- This is the difference between the actual reward ($r + \text{Gamma} * V(s')$) and the expected reward $V(s)$ multiplied by the learning rate α .

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right]$$

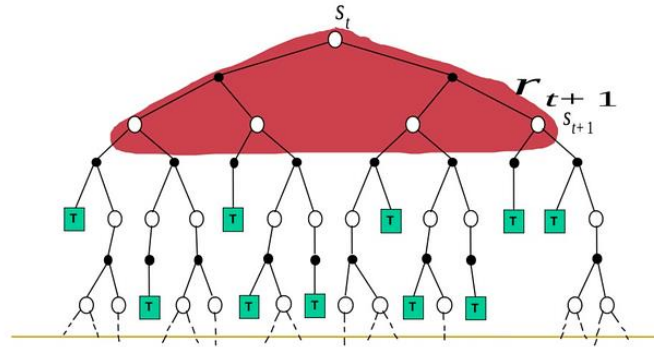
Value of a previous state = value of previous state + learning_rate * (reward + discount_factor(value of current state) — value of previous state)

- Since we take the difference between the actual and predicted values, this is like an error. We can call it a TD error. Iteratively, we will try to minimize this error.

TD Learning: DP vs MC vs TDL

Dynamic Programming

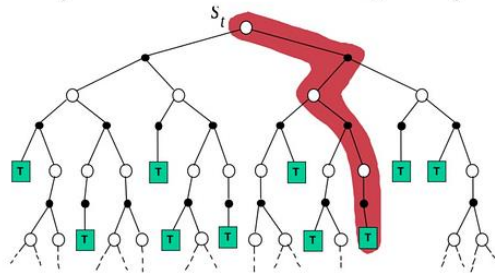
$$V(s_t) \leftarrow E_{\pi} \{r_{t+1} + \gamma V(s_{t+1})\}$$



Monte Carlo Learning

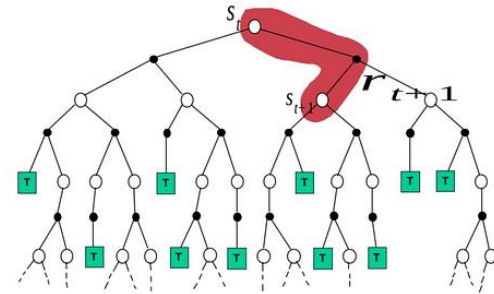
$$V(s_t) \leftarrow V(s_t) + \alpha [R_t - V(s_t)]$$

where R_t is the actual return following state s_t .



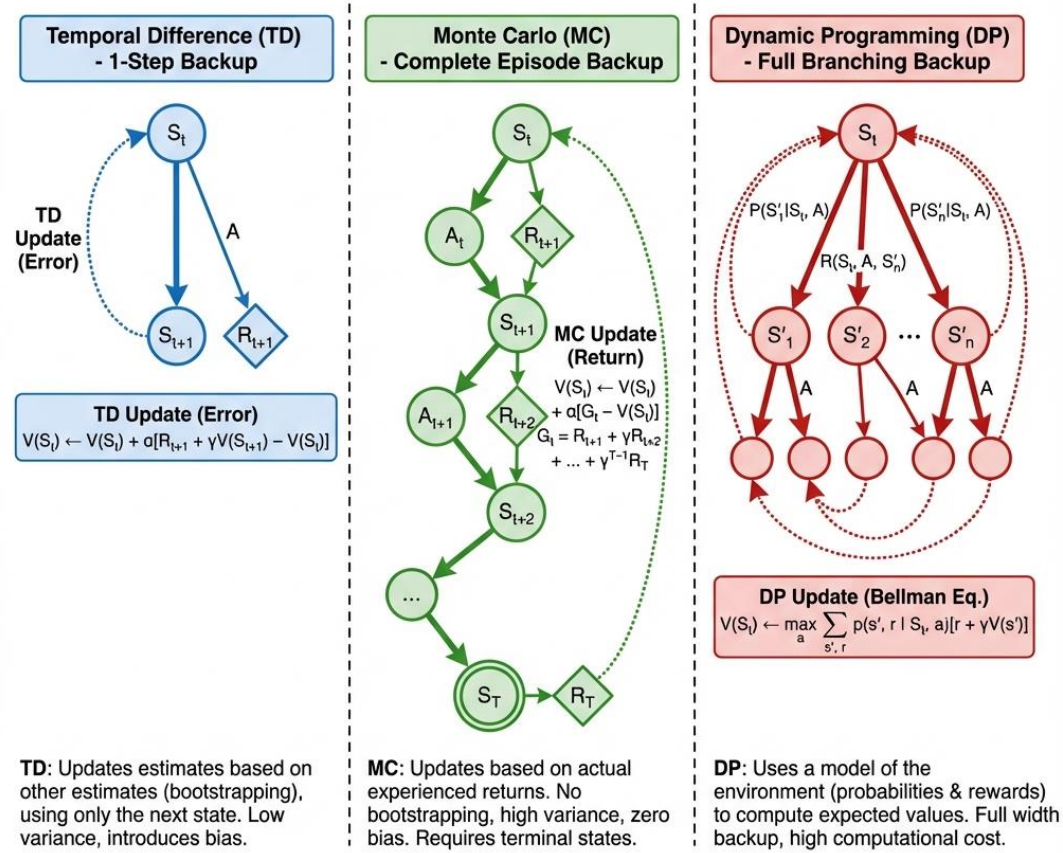
Temporal Difference Learning

$$V(s_t) \leftarrow V(s_t) + \alpha [r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$



TD Learning: DP vs MC vs TDL

Reinforcement Learning Backup Diagrams: TD vs. MC vs. DP



- TD Learning (1-Step, Bootstrapping, Model-Free)
- Monte Carlo (Full Episode, No Bootstrapping, Model-Free)
- Dynamic Programming (Full Branching, Model-Based)

Temporal Difference Prediction

TD Learning: Prediction Algorithm

1. **Prediction & Error:** An agent in state s_t takes an action, receives reward r_{t+1} , and moves to state s_{t+1} . It calculates a **Temporal Difference (TD) Error**: the difference between its new estimate (immediate reward + discounted future value of s_{t+1}) and its old estimate for s_t .
2. **Bootstrapping:** Instead of waiting for the final outcome (like Monte Carlo), TD uses the *estimated* value of the next state ($V(s_{t+1})$) to update the current state's value ($V(s_t)$).
3. **Update Rule:** The value function $V(s)$ is updated using the formula:
$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)].$$
 - α (alpha): Learning rate (how much to update).
 - r : Immediate reward.
 - γ (gamma): Discount factor (importance of future rewards).
 - $V(s')$: Value of the next state.
4. **Incremental Learning:** This process repeats at each time step, gradually refining predictions and improving the agent's policy (decision-making strategy).

TD Learning: Frozen Lake Example

The Frozen Lake problem is a classic reinforcement learning environment used to demonstrate temporal difference (TD) learning algorithms. The agent must learn to navigate a grid from a starting point (S) to a goal (G) by avoiding holes (H) in the slippery ice (F).

	1	2	3	4
1	S	F	F	F
2	F	H	F	H
3	F	F	F	H
4	H	F	F	G

State	Value
(1,1)	0
(1,2)	0
(1,3)	0
...	0
(4,4)	0

- We are in a starting state (s) (1,1), and we take an action right and move to the next state (s') (1,2) and receive a reward (r) of -0.4.

TD Learning: Frozen Lake Example

We update the Value of the state, recalling the TD rule:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- Let us consider a learning rate (α) of 0.1 and a discount factor (γ) of 0.5. Now, substitute the values in the TD rule

$$V(s) = 0 + 0.1 [-0.4 + 0.5 (0) - 0]$$

$$V(s) = ?$$

		Right			
		1	2	3	4
1	S		F	F	F
2	F		H	F	H
3	F		F	F	H
4	H		F	F	G

TD Learning: Frozen Lake Example

We update the Value of the state, recalling the TD rule:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- Let us consider a learning rate (α) of 0.1 and a discount factor (γ) of 0.5. Now, substitute the values in the TD rule

$$V(s) = 0 + 0.1 [-0.4 + 0.5 (0) - 0]$$

$$V(s) = -0.04$$

		Right			
		1	2	3	4
1	S	F	F	F	F
2	F	H	F	H	H
3	F	F	F	H	H
4	H	F	F	G	G

State	Value
(1,1)	-0.04
(1,2)	0
(1,3)	0
...	0
(4,4)	0

TD Learning: Frozen Lake Example

- Now, we are in the state (s) as (1,2) we take an action right and move to the next state (s') (1,3) and receive a reward (r) -0.4.
- learning rate (α) of 0.1 and a discount factor (γ) of 0.5. Now, substitute the values in the TD rule

$$V(s) = 0 + 0.1 [-0.4 + 0.5(0) - 0]$$

$$V(s) = ?$$

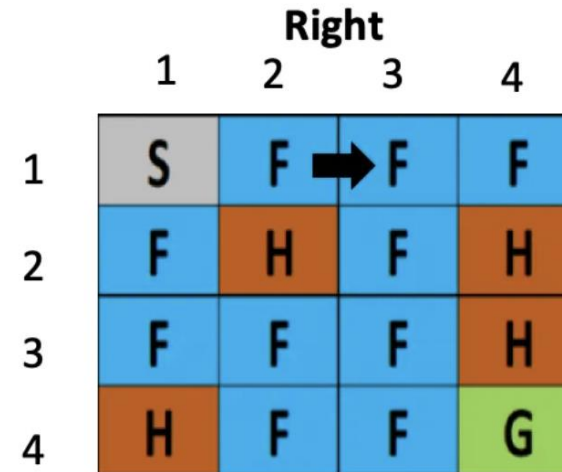
		Right			
		1	2	3	4
1	S	F	F	F	F
2	F	H	F	H	H
3	F	F	F	H	H
4	H	F	F	G	G

TD Learning: Frozen Lake Example

- Now, we are in the state (s) as (1,2) we take an action right and move to the next state (s') (1,3) and receive a reward (r) -0.4.
- learning rate (α) of 0.1 and a discount factor (γ) of 0.5. Now, substitute the values in the TD rule

$$V(s) = 0 + 0.1 [-0.4 + 0.5(0) - 0]$$

$$V(s) = -0.04$$



State	Value
(1,1)	-0.04
(1,2)	-0.04
(1,3)	0
...	0
(4,4)	0

TD Learning: Frozen Lake Example

- Now, we are in the state (s) (1,3). Let's take an action left. We again go back to that state (s') (1,2) and we receive a reward (r) -0.4.
- learning rate (α) of 0.1 and a discount factor (γ) of 0.5. Now, substitute the values in the TD rule

$$V(s) = 0 + 0.1 [-0.4 + 0.5 (-0.04) - 0]$$

$$V(s) = ?$$

Left

	1	2	3	4
1	S	F ←	F	F
2	F	H	F	H
3	F	F	F	H
4	H	F	F	G

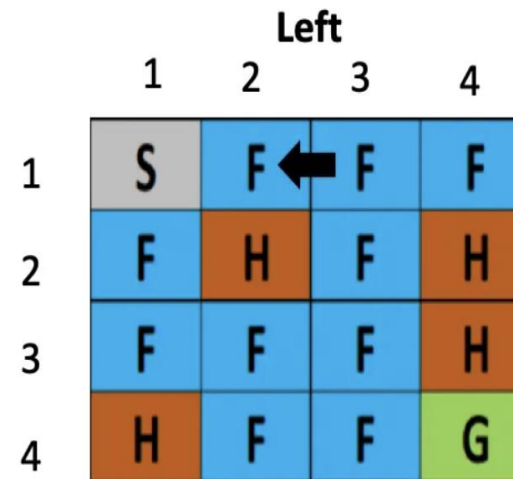
TD Learning: Frozen Lake Example

- Now, we are in the state (s) (1,3). Let's take an action left. We again go back to that state (s') (1,2) and we receive a reward (r) -0.4.
- learning rate (α) of 0.1 and a discount factor (γ) of 0.5. Now, substitute the values in the TD rule

$$V(s) = 0 + 0.1 [-0.4 + 0.5 (-0.04) - 0]$$

$$V(s) = 0.1[-0.42]$$

$$V(s) = -0.042$$



State	Value
(1,1)	-0.04
(1,2)	-0.04
(1,3)	-0.042
...	0
(4,4)	0

TD Learning: Frozen Lake Example

- We update the value of all the states in a similar fashion using the TD update rule. To summarize, the steps involved in the TD prediction algorithm are:
 1. First, initialize $V(S)$ to 0 or some arbitrary value.
 2. Then, begin the episode. For every step in the episode, perform an action A in the state S and receive a reward R and move to the next state (s').
 3. Update the value of the previous state using the TD update rule.
 4. Repeat steps 2 and 3 until we reach the terminal state.

TDL Categories: one-step TD

TD(0) one-step TD (bootstraps immediately): Update the value of the current state using one-step lookahead.

- For policy evaluation (TD prediction):
- TD target:

$$r_{t+1} + \gamma V(S_{t+1})$$

- TD error (delta):

$$\delta_t = r_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

- Update:

$$V(S_t) \leftarrow V(S_t) + \alpha \delta_t$$

- Key properties
 - Online (updates every step)
 - Uses bootstrapping (depends on current estimate $V(S_{t+1})$)
 - Lowest variance among TD family, but can be more biased than longer-step returns

TDL Categories: n-step TD

Instead of one step, look ahead n steps, sum rewards for n steps, and then bootstrap from $V(S_{t+n})$.

- n-step return target:

$$G_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V(S_{t+n})$$

- Update:

$$V(S_t) \leftarrow V(S_t) + \alpha \left(G_t^{(n)} - V(S_t) \right)$$

- Key properties

- Still bootstraps, but after n steps.
- As n increases, it resembles Monte Carlo more.
- Requires waiting for n steps of experience before updating S_t
- Special case: if n equals “until the episode ends,” then $G_t^{(n)}$ becomes the Monte Carlo return (no bootstrapping at all).

TDL Categories: Lambda-return TD(λ)

Combine all n-step targets (1-step, 2-step, 3-step, ...) into a single target using a weighting parameter λ .

- λ -return (conceptual): weighted average of n-step returns

$$G_t^{(\lambda)} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

(In episodic tasks the sum is finite.)

- Key properties
 - Smoothly interpolates between TD(0) and Monte Carlo:
 - $\lambda = 0 \rightarrow$ behaves like TD(0)
 - $\lambda \rightarrow 1 \rightarrow$ approaches Monte Carlo
 - Implemented efficiently via eligibility traces (TD(λ), SARSA(λ), etc.)

Remember:

Bellman Equation: The foundation for TD updates.

For a state value function:

$$V(s) = \mathbb{E}[R_{t+1} + \gamma V(s_{t+1}) \mid s_t = s]$$

for Action-Value Function (Q-function)

$$Q(s, a) = \mathbb{E}_{\pi}[R_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) \mid s_t = s, a_t = a]$$

Temporal Difference (TD) Error: $\delta_t = R_{t+1} + \gamma V(s_{t+1}) - V(s_t)$

TD Learning Update Rule: $V(s_t) \leftarrow V(s_t) + \alpha \delta_t$

Expanded TD Update (in one step): The most basic TD algorithm is TD(0), which updates the value function $V(s)$ for state s as follows:

$$V(s_t) \leftarrow V(s_t) + \alpha [R_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$$

Temporal Difference Control

TD Control: Policy Optimization

In Temporal Difference prediction, we estimated the value function. In TD control, we optimize the value function. There are two kinds of algorithms we use for TD Control:

1. On-policy learning algorithm: SARSA (State-Action-Reward-State-Action)
 2. Off-policy learning algorithm: Q-learning
- On-policy learning: The agent learns about policy π from experience sampled from the same policy π - **learning from its own current actions.**
 - Off-policy learning: The agent learns about policy π from experience sampled from another policy μ - **allowing learning from past experiences or other agents.**

Off-Policy Learning: Q-learning

Q-learning is a model-free RL algorithm where an AI agent learns optimal actions in an environment through trial and error, maximizing long-term rewards by updating "Q-values" (quality) for taking specific actions in specific states, essentially learning what action is best in any given situation to get the most benefits over time.

- We will update the Q value based on the following equation:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- It is an off-policy TD control algorithm. This tells us how good an action is for the agent at a particular state ($Q(s,a)$), rather than looking only at how good it is to be in that state ($V(s)$).

Q-learning: Key Concepts

- **Model-Free:** The agent learns directly from experience without needing a pre-built model of the environment's rules
- **Reinforcement Learning:** The agent learns by receiving rewards (positive) or penalties (negative) for its actions, aiming to maximize cumulative rewards.
- **Q-Table:** A lookup table mapping states to actions, storing the expected future reward (Q-value) for each state-action pair.
- **Q-Value:** The "quality" of taking an action in a state, representing the total discounted future reward.
- **Epsilon-Greedy Strategy:** A method for decision-making: most of the time (greedy), choose the best known action; occasionally (epsilon), explore a random action to discover new possibilities.

Q-learning: Algorithm

- **Initialize:** Create a Q-table with all Q-values set to zero or small random numbers.
- **Observe:** The agent observes its current state (e.g., position in a maze).
- **Act:** Using the epsilon-greedy strategy, the agent chooses an action (e.g., move up, down, left, right).
- **Experience:** The agent performs the action, moves to a new state, and receives a reward (or penalty).
- **Update:** The Q-value for the previous state-action pair is updated using the received reward and the estimated best future reward from the new state (based on Bellman's equation).
- **Repeat:** Steps 2-5 are repeated until the Q-table converges, meaning the agent has learned the optimal policy (best actions).

Q-learning: Frozen Lake Example

The Frozen Lake problem is a classic reinforcement learning environment used to demonstrate temporal difference (TD) learning algorithms. The agent must learn to navigate a grid from a starting point (S) to a goal (G) by avoiding holes (H) in the slippery ice (F).

- Now, let's go back to our example of Frozen Lake. First, initialize the Q function to some arbitrary value.
- Let us say we are in a state (3,2) and have two actions (left and right). Refer to the below figure:

	1	2	3	4
1	S	F	F	F
2	F	H	F	H
3	F	F	F	H
4	H	F	F	G

State	Action	Value
(3,2)	Left	0.2
(3,2)	Right	0.4

Q-learning: Frozen Lake Example

- We select an action using the epsilon-greedy policy in Q-learning. We either explore a new action with probability epsilon or select the best action with probability $1 - \epsilon$. Suppose we select a probability epsilon and select a particular action (moving down):

	1	2	3	4
1	S	F	F	F
2	F	H	F	H
3	F	F	F	H
4	H	F	F	G

State	Action	Value
(3,2)	Left	0.2
(3,2)	Right	0.4
(3,2)	Down	0.6
(4,2)	Up	0.2
(4,2)	Down	0.4
(4,2)	Right	0.6

Q-learning: Frozen Lake Example

- We have performed a downward action in the state (3,2) and reached a new state (4,2) using the epsilon-greedy policy. How do we update the value of the previous state (3,2) using our update rule?
- Let us consider alpha as 0.1, the discount factor as 1 and reward as 0.4:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- $Q((3,2) \text{ down}) = Q((3,2) \text{ down}) + 0.1 (0.4 + 1 \max [Q((4,2) \text{ action})] - Q((3,2), \text{down}))$
- Value of a state (3,2) with downward action is 0.6 in the Q-table.

$\max Q((4,2), \text{action})$ for the state (4,2) =?

Q-learning: Frozen Lake Example

- Greedy policy: there are three actions (up, down, and right), so we will take the maximum value among them.
- Based on the previous Q table, we substitute the values:
 - $Q((3,2), \text{down}) = 0.6 + 0.1 (0.4 + 1 * \max [0.2, 0.4, 0.6] - 0.6)$
 - $Q((3,2), \text{down}) = ?$

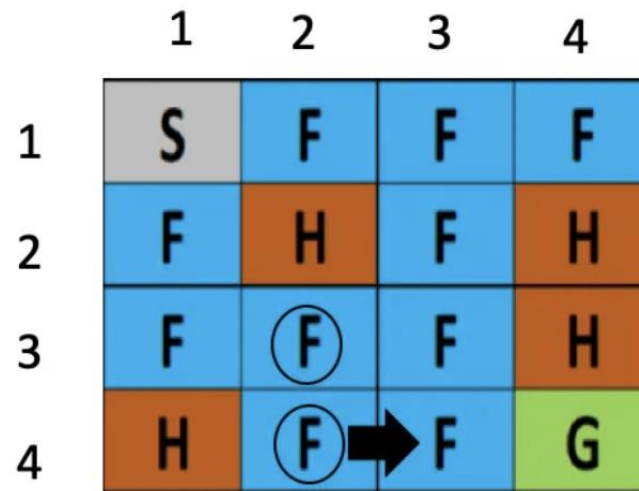
Q-learning: Frozen Lake Example

- Greedy policy: there are three actions (up, down, and right), so we will take the maximum value among them.
- Based on the previous Q table, we substitute the values:
 - $Q((3,2), \text{down}) = 0.6 + 0.1 (0.4 + 1 * \max [0.2, 0.4, 0.6] - 0.6)$
 - $Q((3,2), \text{down}) = 0.64$

So, we update the value of $Q((3,2), \text{down})$ to 0.64.

Q-learning: Frozen Lake Example

- Now, we are in the (4,2) state. What action should we perform? Based on the epsilon-greedy policy, we can either explore a new action with a probability epsilon or select the best action with a probability $1 - \epsilon$. Let us say we select the latter option. So, in (4,2), the action right has a maximum value and that's what we'll select.



State	Action	Value
(4,2)	Up	0.2
(4,2)	Down	0.4
(4,2)	Right	0.6

- Right, we have moved to the state (4,3).

Q-learning: Frozen Lake Example

- Update the value of the previous state?

$$Q((4,2), \text{right}) = Q((4,2), \text{right}) + 0.1 (0.4 + 1 * \max [Q((4,3) \text{ action})] - Q((4,2), \text{right}))$$

- If you look at the Q-table, we have explored only two actions (up and down) for the state (4,3). So, we will take a maximum value based only on these actions (we will not perform an epsilon-greedy policy here; we simply select the action which has the maximum value):

- | | | | | |
|---|---|---|---|---|
| | 1 | 2 | 3 | 4 |
| 1 | S | F | F | F |
| 2 | F | H | F | H |
| 3 | F | F | F | H |
| 4 | H | F | F | G |

State	Action	Value
(4,2)	Up	0.2
(4,2)	Down	0.4
(4,2)	Right	0.6
(4,3)	Up	0.2
(4,3)	Down	0.4

Q-learning: Frozen Lake Example

$$Q((4,2), \text{right}) = Q((4,2), \text{right}) + 0.1 (0.4 + 1 \max [(Q(4,3), \text{up}), (Q(4,3), \text{down})] - Q((4,2), \text{right}))$$

$$\begin{aligned} Q((4,2), \text{right}) &= 0.6 + 0.1 (0.4 + 1 \max [0.2, 0.4] - 0.6) \\ &= 0.6 + 0.1 (0.4 + 1(0.4) - 0.6) \\ &= 0.62 \end{aligned}$$

- Awesome! We'll update the value of the state $Q((4,2), \text{right})$ to 0.62.

•

On-Policy Learning: SARSA

- SARSA (State-Action-Reward-State-Action) is a fundamental on-policy RL algorithm where an agent learns to make decisions by following a sequence of its own experiences.
- We will update the Q-value based on the following equation:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- You might have noticed that there is no $\max Q(s', a')$ (unlike Q-learning).
- It updates its action-value (Q-value) estimates only based on the actual policy being followed (including exploration), making it safer in environments where the path matters.

SARSA: Key Concepts

- **S (State)**: The agent's current situation or position in the environment (e.g., a square on a grid).
- **A (Action)**: The decision the agent makes in that state (e.g., move up, down, left, right).
- **R (Reward)**: The feedback (positive or negative) received after the action.
- **S' (Next State)**: The new state the agent enters.
- **A' (Next Action)**: The action the agent chooses in the *next* state, according to its current policy.

SARSA: Algorithm

- **Observe (S, A):** Agent is in state S, takes action A.
- **Act & Get Feedback:** Agent receives reward R, moves to state S'.
- **Choose Next Action (A'):** Agent selects the next action A' in state S' based on its current policy (e.g., epsilon-greedy).
- **Update Q-Value:** The Q-value for (S, A) is updated using the reward R and the Q-value of the next (S', A') pair.
- **Repeat:** This process continues, refining the agent's understanding of good actions for each state.

SARSA: Frozen Lake Example

The Frozen Lake problem is a classic reinforcement learning environment used to demonstrate temporal difference (TD) learning algorithms. The agent must learn to navigate a grid from a starting point (S) to a goal (G) by avoiding holes (H) in the slippery ice (F).

- Suppose we are in state (4,2). We decide on the action using the epsilon-greedy policy.
- Let's say we use a probability $1 - \epsilon$ and select the best action (moving right):

	1	2	3	4
1	S	F	F	F
2	F	H	F	H
3	F	F	F	H
4	H	F	F	G

State	Action	Value
(4,2)	Up	0.2
(4,2)	Down	0.4
(4,2)	Right	0.6

SARSA: Frozen Lake Example

- We land in the state (4,3). How do we update the value of the previous state (4,2)?
- Let us consider the alpha as 0.1, the reward as 0.4, and the discount factor as 1.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

$$Q((4,2), \text{right}) = Q((4,2), \text{right}) + 0.1 (0.4 + 1 * Q((4,3), \text{action})) - Q((4,2), \text{right})$$

- How do we choose the value for $Q((4,3), \text{action})$?
- We can't just pick up $\max (Q(4,3), \text{action})$ as we did in Q-learning.

SARSA: Frozen Lake Example

- In SARSA, we use the epsilon-greedy policy. Look at the Q table I've shown below. In state (4,3), we have explored two actions.

	1	2	3	4
1	S	F	F	F
2	F	H	F	H
3	F	F	F	H
4	H	F	F	G

State	Action	Value
(4,2)	Up	0.2
(4,2)	Down	0.4
(4,2)	Right	0.6
(4,3)	Up	0.2
(4,3)	Down	0.4

- We either explore with a probability epsilon or exploit with a probability $1 - \epsilon$. Let's say we select the former option and explore a new action (moving right).

SARSA: Frozen Lake Example

	1	2	3	4
1	S	F	F	F
2	F	H	F	H
3	F	F	F	H
4	H	F	F	G

State	Action	Value
(4,2)	Up	0.2
(4,2)	Down	0.4
(4,2)	Right	0.6
(4,3)	Up	0.2
(4,3)	Down	0.4
(4,3)	Right	0.7

$$Q((4,2), \text{right}) = Q((4,2), \text{right}) + 0.1 (0.4 + 1 (Q(4,3), \text{right}) - Q((4,2), \text{right}))$$

$$Q((4,2), \text{right}) = 0.6 + 0.1 (0.4 + 1(0.7) - 0.6)$$

$$Q((4,2), \text{right}) = 0.65$$

- And that's how we get the state-action values in SARSA. We take the action using the epsilon-greedy policy, pick the action, and update the Q value.

n-Step Bootstrapping

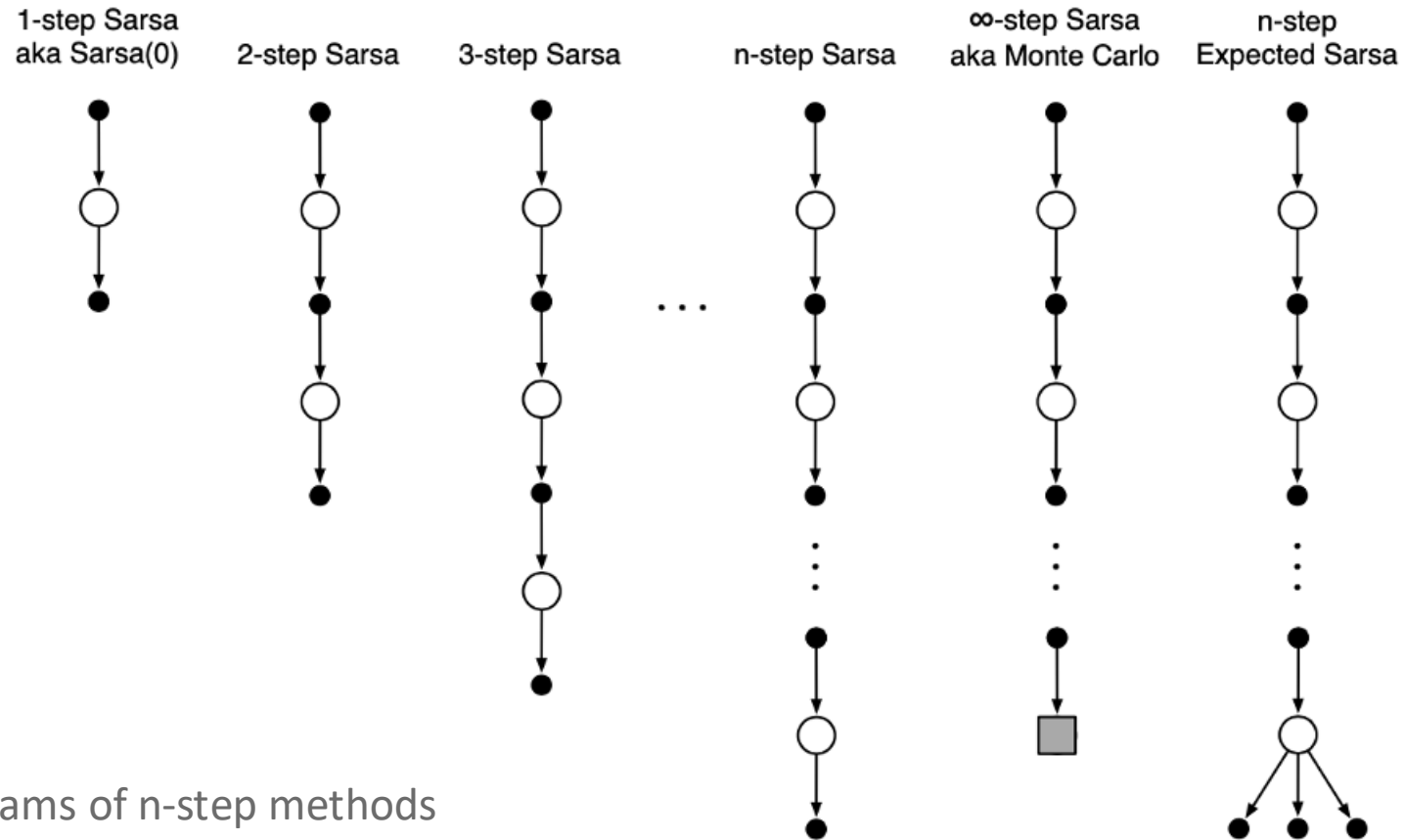
n-Step Bootstrapping

- n-step bootstrapping is a Reinforcement Learning technique that bridges Monte Carlo (MC) methods (waiting for the whole episode) and one-step Temporal Difference (TD) learning (using only the next step) by using an intermediate number of steps, n , to estimate the return, balancing learning speed and variance.
- It calculates the return using rewards from the next n steps plus the bootstrapped value estimate of the state reached at step n , providing a flexible way to update value functions by generalizing TD and MC approaches.

n-Step Bootstrapping

- Let's understand what one-step TD and MC algorithms have in common. If we omit overly specific details, we will notice that both of them are actually very similar and use the same state update rule except for a single difference:
 - One-step TD updates every state by looking at the $n = 1$ next state.
 - MC updates every state after analyzing the whole episode. This can be roughly viewed as an update for every state that occurs after n steps (where n can be a large number).

n-Step Bootstrapping



The backup diagrams of n-step methods

n-Step Bootstrapping

- We have two extreme cases: $n = 1$ and $n = \text{the number of remaining states in the episode sequence}$.
- A logical question arises, could we use a value of n that falls somewhere in the middle of these extreme values?

The answer is yes. And this concept is generalized through n-step Bootstrapping.



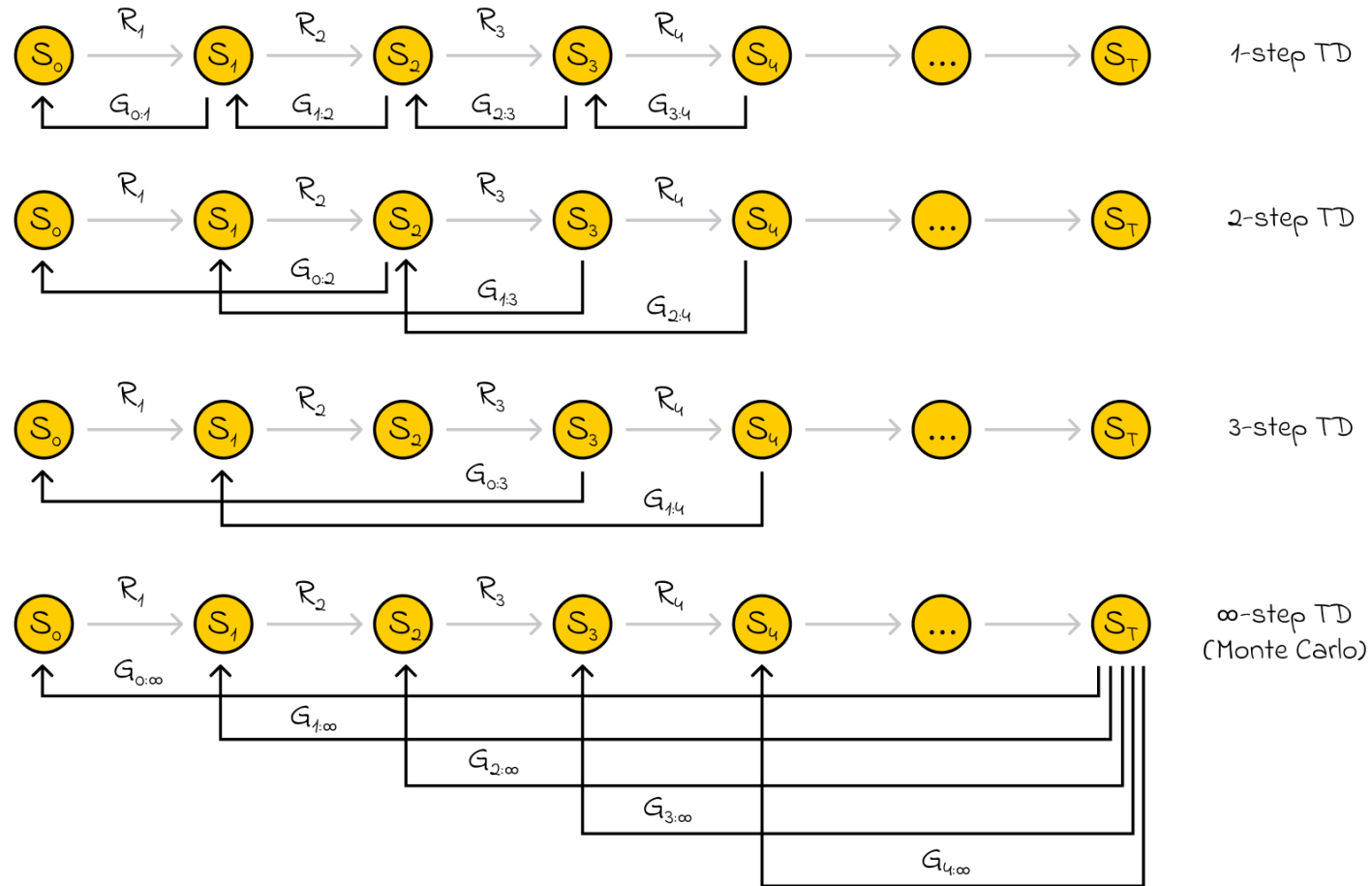
n-Step Bootstrapping: Workflow

- In one-step TD, we analyze the difference between the received reward and how the state value changes by switching from the current state to the next ($n = 1$). This idea can be easily generalized to multiple steps.
- All n-step returns can be considered approximations to the full return, truncated after n-steps, and corrected for the remaining missing terms.
- Update rule for n-step Bootstrapping

$$G_{t:t+n} \doteq R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n}),$$

$$V(S_t) = V(S_t) + \alpha(G_{t:t+n} - V(S_t))$$

n-Step Bootstrapping: Workflow



Relationships between rewards and state values during updates for different values of n

n-Step Bootstrapping: Workflow

For instance, let us take $n = 3$;

- The beginning of the episode is generated up until the state S_3 .
- State S_0 is updated using the 3-step return, which sums rewards R_1 , R_2 , and R_3 and the value of state S_3 .
- State S_4 is generated.
- State S_1 is updated by using the 3-step return, which sums up rewards R_2 , R_3 and R_4 and the value of state S_4 .
- State S_5 is generated.
- State S_2 is updated using the 3-step return, which sums rewards R_3 , R_4 , and R_5 and the value of state S_5 .
- The analogous process is repeated until we reach the last state of the episode.

n-Step Bootstrapping: Workflow

- If for a given state, there are fewer than n states left to calculate the n -step return, then the **truncated n -step return** is used instead, accumulating available rewards until the terminal state.
- If $n = \infty$, then n -step TD is a Monte Carlo algorithm

n-Step Bootstrapping: Workflow

n-step TD for estimating $V \approx v_\pi$

Input: a policy π

Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n

Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$

All store and access operations (for S_t and R_t) can take their index mod $n + 1$

Loop for each episode:

 Initialize and store $S_0 \neq \text{terminal}$

$T \leftarrow \infty$

 Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take an action according to $\pi(\cdot | S_t)$

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then $T \leftarrow t + 1$

$\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)

 If $\tau \geq 0$:

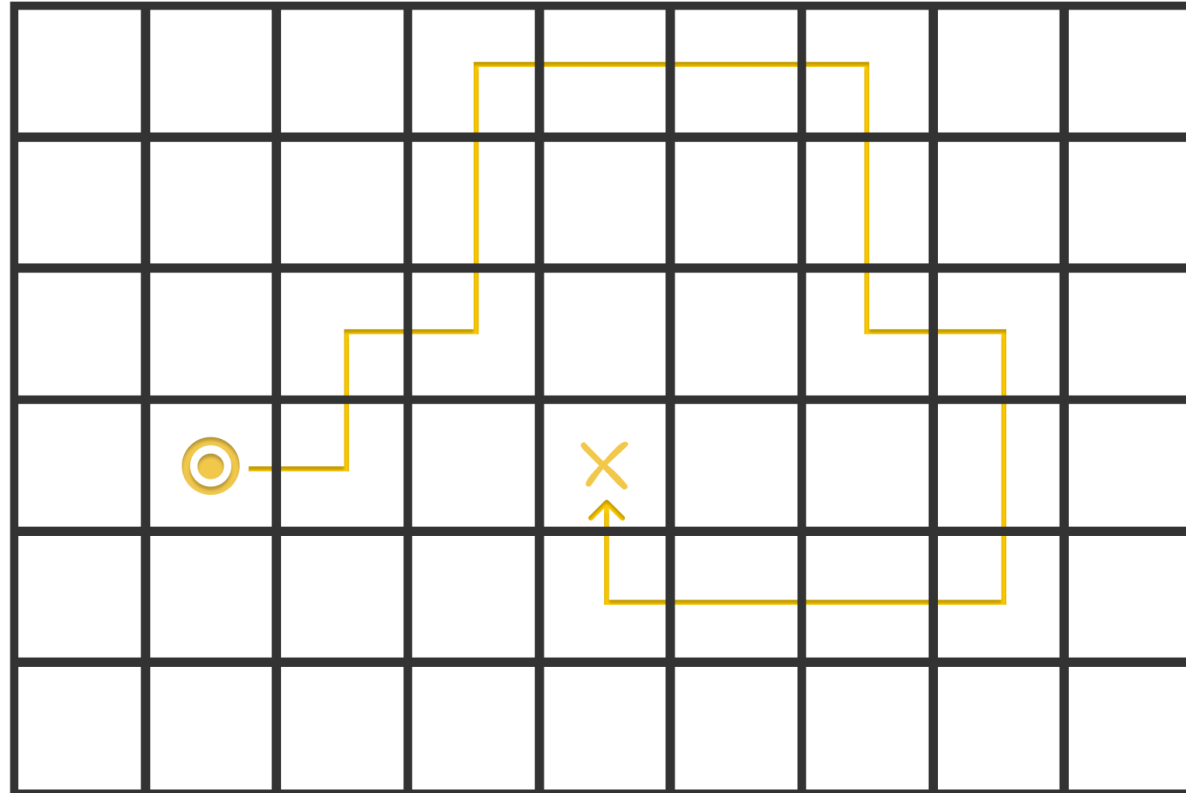
$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ ($G_{\tau:\tau+n}$)

$V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$

 Until $\tau = T - 1$

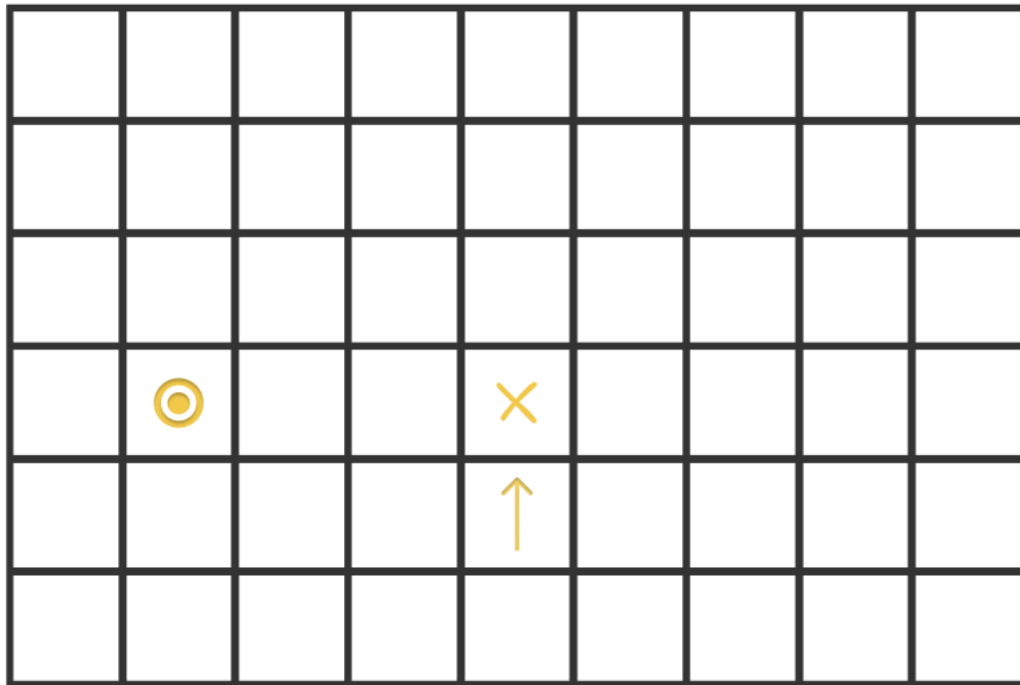
n-Step Bootstrapping: Choosing $n=?$



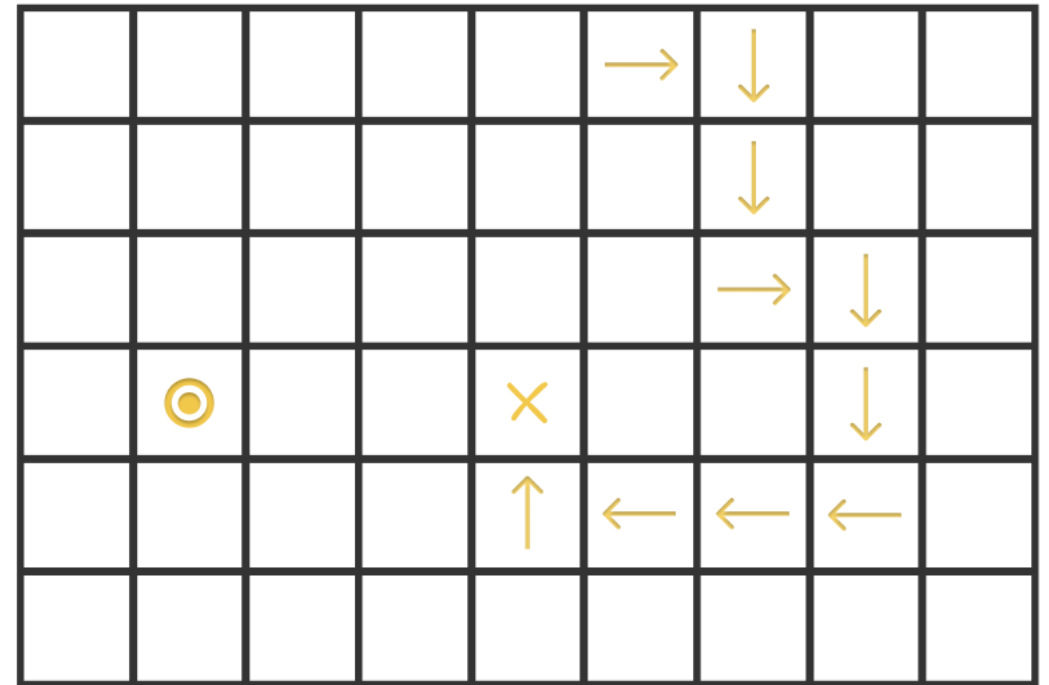
The path made by the agent in the maze. The X mark represents the terminal state

Source: [Reinforcement Learning. An Introduction. Second Edition | Richard S. Sutton and Andrew G. Barto](#)
23-01-2026

n-Step Bootstrapping: Choosing $n=?$



1-step Sarsa



10-step Sarsa

Visualisation of action states whose values were modified in both algorithms during the first episode. As we can see, 10-step Sarsa learns much faster than 1-step Sarsa.

Source: [Reinforcement Learning. An Introduction. Second Edition | Richard S. Sutton and Andrew G. Barto](#)

23-01-2026

Models and Planning

Models

- Distribution Model
 - The models that produce a description of all possibilities and their probabilities; these we call distribution models.
- Sample Model
 - The models that produce just one of the possibilities, sampled according to the probabilities; these we call sample models

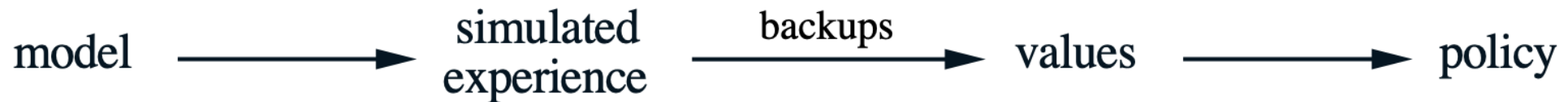


Planning

- State-Space Planning
 - It includes the approach we take in this book, is viewed primarily as a search through the state space for an optimal policy or an optimal path to a goal. Actions cause transitions from state to state, and value functions are computed over states.
- Plan-Space Planning
 - Plan-space planning, planning is instead a search through the space of plans. Operators transform one plan into another, and value functions, if any, are defined over the space of plans. Plan-space planning includes evolutionary methods and “partial-order planning,” a common kind of planning in artificial intelligence in which the ordering of steps is not completely determined at all stages of planning.

Planning: State-Space Planning

- There are two basic ideas: (1) all state-space planning methods involve computing value functions as a key intermediate step toward improving the policy, and (2) they compute value functions by updates or backup operations applied to simulated experience. This common structure can be diagrammed as follows:



- Dynamic programming methods clearly fit this structure: they make sweeps through the state space, generating, for each state, the distribution of possible transitions. Each distribution is then used to compute a backed-up value (update target) and update the state's estimated value.

Dyna Algorithm

- Dyna is an algorithm that efficiently combines direct RL (model-free) learning from real experience with model-based planning, allowing agents to learn faster by simulating experiences using a learned model of the environment, integrating learning, planning, and reactive execution for improved sample efficiency and faster convergence.

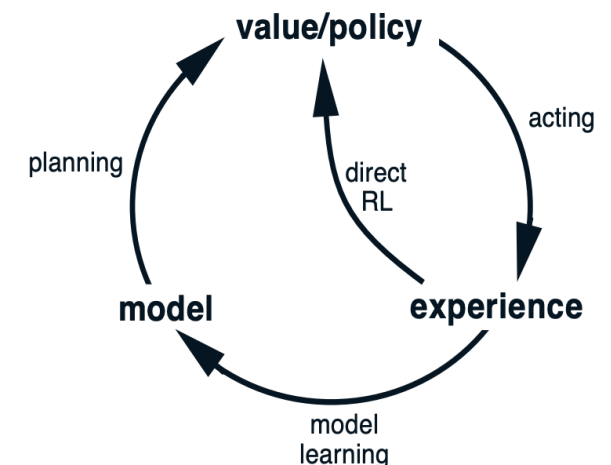
Dyna is a family of reinforcement-learning architectures (introduced by Sutton) that tightly couples three things in one loop:

- **Acting** (interact with the real environment)
- **Learning** (update value function / Q-values from real experience)
- **Planning** (use a **model**-learned or given-to simulate additional experience and update values again)
- The core idea is simple: **every real transition you experience can be reused many times** by “imagining” similar transitions through a model, so learning becomes much more data-efficient.

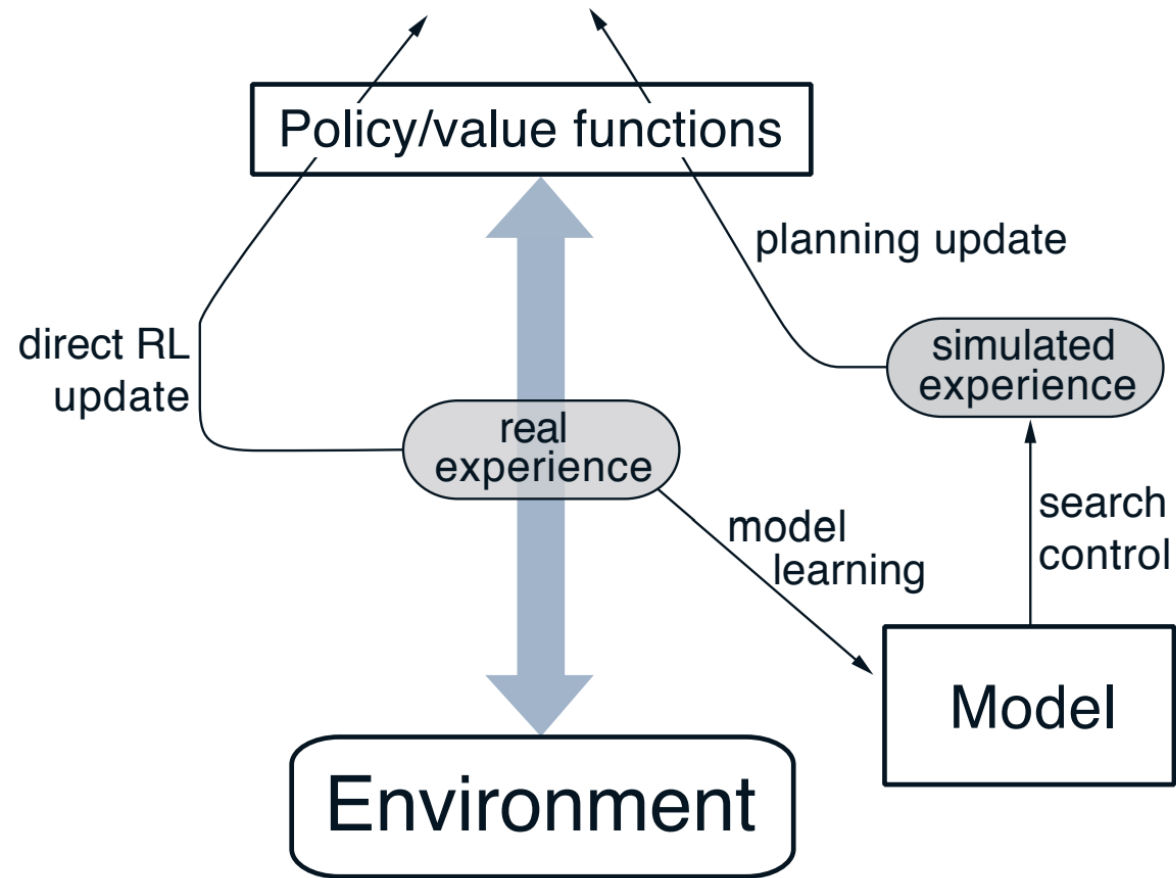
Dyna Algorithm: Workflow

- **Model Learning:** The agent uses its interactions with the environment to learn a model that approximates the dynamics of the environment.
- **Planning:** The agent performs planning using the learned model to simulate future states and rewards. It uses algorithms, such as DP or MC simulations, to update the value function. This allows the agent to evaluate different actions and refine its policy without actually performing them in the environment.
- **Acting:** The agent takes actions in the real environment based on its current policy.

The agent interacts with the environment, updates its model, performs planning using the model, and adjusts its policy based on the learned values



Dyna Algorithm: Workflow



The general Dyna Architecture. Real experience, passing back and forth between the environment and the policy, affects policy and value functions in much the same way as does simulated experience generated by the model of the environment.

Dyna Algorithm: Workflow

Tabular Dyna-Q

Initialize $Q(s, a)$ and $Model(s, a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$

Loop forever:

- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$ -greedy(S, Q)
- (c) Take action A ; observe resultant reward, R , and state, S'
- (d) $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
- (e) $Model(S, A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
 - $S \leftarrow$ random previously observed state
 - $A \leftarrow$ random action previously taken in S
 - $R, S' \leftarrow Model(S, A)$
 - $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

Dyna Algorithm: Stepwise Workflow

Step 1: Initialize the state action values and the model (environment dynamics) randomly.

Step 2: Model Learning (Direct RL):

- The agent interacts with the environment by selecting actions and observing state transitions and rewards.
- For example, if the agent takes the action 'right' from state (1, 1), it observes that it transitions to state (1, 2) with a reward of 0.
- The agent updates its model based on these observed state transitions and rewards.

Step 3: Planning:

Planning has also 3 components:

Search control:

- It selects a previously visited state action pair at random
- Let's say during planning, the agent simulates taking the action 'up' from state (2, 2) and observes that it reaches the goal state with a reward of 1.

Dyna Algorithm: Stepwise Workflow

Model query:

- Using the learned model, the agent performs planning to simulate or predict future states and rewards.

Value updates:

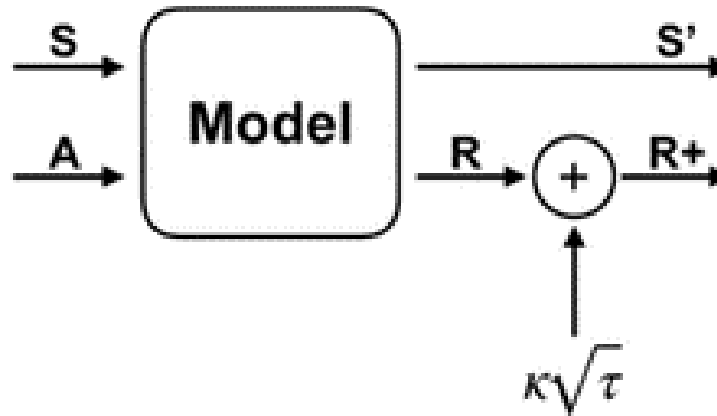
- Based on this simulated experience, the agent knows the S, A, R, S, A .
- The agent can use an algorithm like MC, TD or DP to update its value function.

Step 4: Acting:

- The agent selects actions to take in the real environment based on its current policy.
- Initially, the policy might be random or exploratory. As the agent collects more real-world experiences and plans, it updates its policy to improve decision-making.

Dyna Q+ Algorithm:

It will update the rewards based on time. If the state was visited by the agent long back. It will increase the reward (by the square root of the time step) which will motivate the agent to visit the state again.



Where τ is the time steps since the transition was last visited and K is the constant to control the amount of weight we want to give. If K is 0, then Dyna Q+ is simple Dyna Q.

Trajectory Sampling

Trajectory Sampling

Trajectory sampling in reinforcement learning involves generating learning data by running the agent through the environment to produce full sequences of experience, called trajectories.

A trajectory is a time-ordered sequence:

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T)$$

For episodic tasks, it ends at a terminal state S_T . For continuing tasks, it can be an ongoing stream.

Trajectory Sampling

- Trajectory Sampling in RL involves strategically selecting sequences of states, actions, and rewards (trajectories) to improve learning efficiency, especially in complex tasks or offline settings where data is limited, by prioritizing informative paths, managing uncertainty, or generating new samples with learned models to find better policies faster.

Trajectory sampling underlies most model-free RL methods:

- Monte Carlo (MC): needs complete trajectories to compute returns (waits for episode end).
- TD learning (TD(0), SARSA, Q-learning): still uses trajectory sampling, but updates at each step rather than waiting for the end.
- Policy Gradient methods: typically use batches of sampled trajectories to estimate gradients.

Trajectory Sampling: Core Paradigm

- **Model-Based Planning (PETS):** The Probabilistic Ensemble with Trajectory Sampling (PETS) framework uses an ensemble of dynamics models to propagate uncertainty. By sampling multiple potential trajectories (particles) for candidate action sequences, PETS allows agents to plan robustly while quantifying both model and environmental uncertainty.
- **Trajectory Replay (TR):** Unlike traditional experience replay that samples random transitions, TR stores and samples entire temporally contiguous sequences. This preserves long-range dependencies and is particularly effective for offline RL, where agents must learn from a fixed dataset without new interactions.
- **Rare Event Sampling:** In physics and safety-critical domains, RL is used to "make rare events typical" by reweighting trajectory ensembles. This helps in simulating atypical but critical behaviors, such as system failures or rare physical excursions

Trajectory Sampling: Example

Suppose the agent starts at $(0, 0)$ and follows actions Right, Right, Down, Down to reach goal.

A sampled trajectory could be:

$$(0, 0) \xrightarrow{R, -1} (0, 1) \xrightarrow{R, -1} (0, 2) \xrightarrow{D, -1} (1, 2) \xrightarrow{D, +10} (2, 2) = G$$

States/actions/rewards depend on the grid definition.
That entire sequence is a trajectory sample.

Trajectory Sampling: Sampling Techniques

- **Prioritized Trajectory Replay (PTR):** Trajectories are prioritized based on metrics like high returns, low model uncertainty, or high Generalized Advantage Estimation (GAE).
- **Backward Sampling:** Propagating reward signals from the end of a trajectory to the beginning in reverse order significantly stabilizes training and accelerates convergence.
- **Generative/Diffusion Sampling:** Modern approaches in 2026 often use Diffusion Models to generate high-quality synthetic trajectories for replay, mitigating "catastrophic forgetting" in continual learning.

Trajectory Sampling: Applications

- **Robotics:** Safe trajectory sampling using Gaussian processes ensures robots respect physical constraints and avoid obstacles without requiring simplified assumptions.
- **Autonomous Vehicles:** Multi-agent systems use trajectory sampling to coordinate decisions and predict future paths of other vehicles to prevent collisions.
- **Reasoning Tasks:** Large Language Models (LLMs) now use trajectory-level Bellman residuals to improve multi-turn reasoning and agentic workflows

Trajectory Sampling: Real Environment

When the agent follows a behaviour policy (often ϵ -greedy) in the actual Grid world, it generates a trajectory:

$$\tau_{real} = (S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T)$$

Example (4×4 Grid world, one episode):

$$(0,0) \xrightarrow{R,-1} (0,1) \xrightarrow{R,-1} (0,2) \xrightarrow{D,-1} (1,2) \xrightarrow{D,-1} (2,2) \xrightarrow{R,+10} (2,3) = G$$

Each step gives one sampled transition (s, a, r, s') .

TD methods (TD(0), SARSA, Q-learning) can update from these samples immediately.

Full backup Planning (DP): not Trajectory Sampling

If you know the full transition model $P(s' | sa)$, a DP backup uses an expectation over all possible next states, not a sampled next state:

$$Q(s, a) \leftarrow \sum_{s'} P(s' | sa) (R(s, a, s') + \gamma \max_{a'} Q(s', a'))$$

This is planning by expectation, not trajectory sampling.

Dyna: trajectory sampling inside planning (simulated trajectories)

Dyna builds/uses a model $\hat{M}$ from real experience:

$$\hat{M}(s, a) \approx (r, s')$$

Then, besides the real TD update, it performs additional planning updates by *sampling* transitions from the model this is trajectory sampling from the model.

Dyna-Q : At each real step:

- Act (real): take action a , observe (s, a, r, s')
- Learn (real TD update):

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- Update model: store $\hat{M}(s, a) = (r, s')$

Dyna: trajectory sampling inside planning (simulated trajectories)

- Plan (simulated TD updates): repeat k times:
 - sample a previously seen $(\tilde{s}, \tilde{a})$
 - get $(\tilde{r}, \tilde{s}') = \hat{M}(\tilde{s}, \tilde{a})$
 - apply the same TD update to $Q(\tilde{s}, \tilde{a})$

This is “trajectory sampling” because the planning phase is generating simulated experience transitions that look like:

$$(\tilde{s}, \tilde{a}, \tilde{r}, \tilde{s}')$$

If you chain multiple simulated steps, you get a simulated trajectory:

$$\tau_{model} = (\tilde{S}_0, \tilde{A}_0, \tilde{R}_1, \tilde{S}_1, \dots)$$

So Dyna uses trajectory sampling twice:

- from the real environment (real trajectories)
- from the learned model (simulated trajectories)